ORIONFLOW JAVA / CODEQL

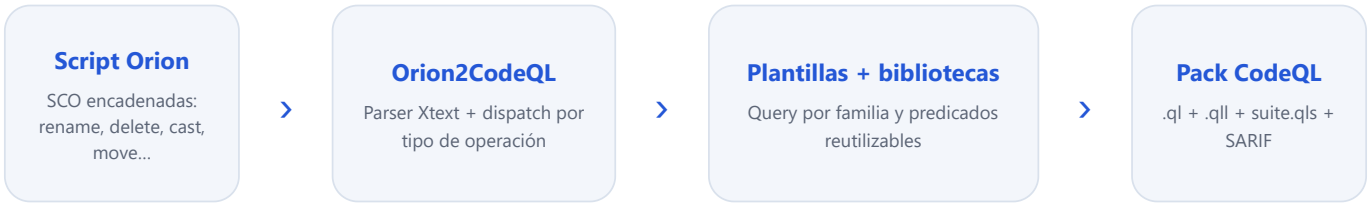
Arquitectura y validación del generador V3.2

Una guía visual de cómo se transforma una evolución Orion en consultas CodeQL y de las capas de prueba usadas para garantizar coherencia sintáctica, semántica y funcional.

35	48/48	2.27.2
tests JUnit superados	queries V3.2 compiladas	versión CodeQL objetivo

1 Vista general de la arquitectura

El generador mantiene separadas tres responsabilidades: comprender la evolución, resolver la identidad original de los elementos y construir consultas CodeQL modulares.



Orquestación

`Orion2CodeQL` conserva el `dispatch generateOperation(...)`. Reconoce cada SCO, consulta el contexto y delega la generación.

Identidad evolutiva

`EvolutionContext` separa nombre actual y nombre original. Así, un rename de entidad seguido de un rename de campo sigue buscando los símbolos que realmente existen en el código.

Generadores de consultas

EntityQuery
Rename, Delete y Split Entity.

FeatureQuery
Rename, Delete y Move Feature.

AttributeQuery
Cast Attribute, Cast Reference y Promote.

RelationshipQuery
Rename y Delete Relationship.

Regla de diseño: las clases Query construyen CodeQL; no gestionan el historial de evolución. El contexto se resuelve antes de delegar.

2 Bibliotecas generadas y perfiles

orion_java.qll

Símbolos Java, constructores, tipos, campos, accessors e invocaciones resueltas.

orion_spring.qll

Reglas comunes Spring: Validator y repositorios.

orion_jpa.qll

JPA, JPQL, Spring Data JPA y namespaces jakarta/javax.

orion_mongo.qll

Documentos, componentes embebidos, Criteria, Sort y nombres persistidos.

orion_dataflow.qll

Flujo local, dirigido por la SCO y con fuentes/sinks restringidos.

orion_callgraph.qll

Propagación resuelta y acotada a uno o dos saltos, con valores y efectos.

utils.qll

Fachada estable que compone solo los adaptadores seleccionados.

Composición por framework

Perfil	Java/Spring	JPA	Mongo	Contrato
`jpa`	✓	✓	—	Valor por defecto.
`mongo`	✓	—	✓	Sin contaminación JPA.
`both`	✓ una vez	✓	✓	Debe equivaler a la unión exacta.

Entradas públicas

- ✓ `-i / --input`
- ✓ `-o / --output`
- ✓ `--framework jpa|mongo|both`
- ✓ `-a ALL` como alias obsoleto sin efecto

Estado V3.2

VALIDADO Call Graph acotado y DataFlow global limitado a CAST forman parte del pack y se han ejecutado con CodeQL 2.27.2.

Los nombres conocidos por el generador se insertan como literales en la query. Se evita construir expresiones regulares con nodos AST como `parent.getName()`, origen de la degradación extrema detectada anteriormente.

3 Estrategia de pruebas por capas

No se considera suficiente que Xtend compile. La validación sube progresivamente desde unidades puras hasta ejecutar las consultas sobre una base CodeQL real.



EJECUTADO

Baseline V3.1

JUnit, generación, compilación QL, base de fixtures, evaluación SARIF y tiempos.

AUTOMATIZABLE

Scripts

`verify-v2.ps1` codifica el patrón integral; `compare-sarif.ps1` calcula métricas contra ground truth.

EJECUTADO

V3.2

Fixtures y regresiones específicas de Call Graph y DataFlow global CAST.

Principio de coherencia

Cada capa comprueba una propiedad distinta. Si una query contiene el texto esperado pero usa una API inexistente, falla la compilación CodeQL. Si compila pero enlaza el símbolo equivocado, fallan los fixtures SARIF. Si los perfiles se mezclan, falla la igualdad de conjuntos.

4 Pruebas unitarias e integración Java

Suite	Tests	Qué garantiza
`PersistenceFrameworkTest`	3	Parseo de jpa/mongo/both, default JPA, capacidades y rechazo de valores no soportados.
`Orion2CodeQLConfigTest`	6	CLI, input/output, framework, ayuda, alias `-a ALL` y rechazo de modos eliminados.
`EvolutionContextTest`	5	Renames encadenados, move, split, IDs estables, tombstones y relaciones rename→delete.
`QueryGenerationTest`	7	Fragmentos estructurales/semánticos, DataFlow selectivo, JoinTable, NamedQuery y APIs compatibles.
`ModuleGenerationTest`	10	Composición, Call Graph acotado, DataFlow local/global selectivo, constantes y ausencia de regex dinámicas.
`Orion2CodeQLIntegrationTest`	4	AST Orion → mapa de ficheros, nombres originales, perfiles, reinicio de estado y EvolBlocks.
Total actual	**35**	**Ejecutados con Maven y JDK 11.**

Unitarios de plantilla

Inspeccionan el QL generado para asegurar que aparece el predicado correcto y que no reaparecen patrones prohibidos. Son rápidos y localizan regresiones en la plantilla.

Integración del generador

Construye modelos Orion reales con EMF, ejecuta el transformador y comprueba nombres de archivos, módulos y resolución histórica.

Qué no demuestran por sí solos

Una aserción textual no demuestra que la API QL exista ni que el predicado encuentre resultados correctos. Por eso estas suites son la base, no el final de la validación.

5 Pipeline de validación extremo a extremo

1. **Build limpio del generador**
   `mvn clean test` con JDK 11. Compila Xtend/Java y ejecuta las 35 pruebas.

2. **Compilación de fixtures**
   El proyecto Java interno debe compilar para que las referencias, overloads y tipos sean resolubles.

3. **Empaquetado y ejecución CLI**
   Se genera el JAR y se ejecuta para default, JPA, Mongo y Both.

4. **Contrato de artefactos**
   Se comprueban módulos obligatorios/prohibidos, número de queries y equivalencia byte a byte default = JPA.

5. **Compilación CodeQL**
   `query compile --check-only` sobre cada `.ql`, usando CLI 2.27.2 y `java-all@9.3.2`.

6. **Base CodeQL real**
   `database create` instrumenta el build Maven de los fixtures.

7. **Ejecución completa**
   `database analyze` produce SARIF para cada perfil sobre la misma base.

8. **Oráculo funcional**
   Normalización, positivos/negativos, ausencia de duplicados y unión exacta de perfiles.

9. **Rendimiento**
   Se separan generación, compilación de planes, evaluación, total y alertas.

Compatibilidad de entorno: Xtend 2.25.0 no puede ejecutarse con bytecode Java 21 en esta configuración. La validación reproducible fija JDK 11; el JAR standalone necesita `--add-opens=java.base/java.lang=ALL-UNNAMED` por Guice/CGLIB.

6 Oráculo SARIF y coherencia entre perfiles

Cada hallazgo se normaliza mediante una identidad estable:

```
(ruleId, URI normalizada, línea/columna inicial y final, mensaje)
```

Positivos obligatorios

- ✓ Regla concreta.
- ✓ Fichero y rango de líneas.
- ✓ Fragmento semántico del mensaje.
- ✓ Símbolo resuelto esperado.

Negativos obligatorios

- → Clases homónimas no persistentes.
- → Strings iguales en APIs arbitrarias.
- → Propiedades de otra entidad.
- → Expresiones dinámicas no resolubles.

Invariantes de composición

Invariante	Fallo que evita
Default = JPA explícito	Cambio silencioso de compatibilidad CLI.
JPA no contiene resultados Mongo	Acoplamiento o import de adaptador indebido.
Mongo no contiene resultados JPA	Contaminación cruzada entre frameworks.
Both = conjunto(JPA ∪ Mongo)	Pérdidas, extras o precedencia lógica incorrecta.
Sin identidades duplicadas	Dos caminos equivalentes reportando el mismo impacto.

> Comparar solo el número total no basta: una pérdida y un falso positivo podrían compensarse. La comparación se realiza sobre conjuntos de identidades normalizadas.

7 Cobertura funcional de fixtures

Capacidad	Positivos	Negativos / control
Constructores Entity	Público vacío, público con parámetro y privado.	Clase Visit homónima no persistente.
EvolutionContext	Rename Entity → Rename Feature → Delete.	No buscar nombres nuevos inexistentes en Java.
Mongo embebido	List, array, anidación y ciclos acotados.	static, transient, DBRef y DocumentReference no crean ownership.
Criteria / Sort	where, and, by, Order.asc y Order.desc.	Variable dinámica y método arbitrario con el mismo literal.
Validator Object	Cast local Object → Visit y supports(Visit).	Cast a otra entidad y método ambiguo.
Constantes	Literal, static final, clase externa y concatenación.	Constante no relacionada y expresión dinámica.
Metadatos	Valor lógico explícito con dependencia real.	Marcadores @Entity/@Table/@Document/Lombok redundantes.
Call Graph V3.2	Uno/dos saltos, mutación indirecta, helper privado y despacho virtual.	Resultado descartado y overload no relacionado.
Global CAST V3.2	Argumentos, retornos, cadenas y varios parámetros del mismo tipo.	Fuente ajena y consumidor compatible con el tipo nuevo.

Resultados reales V3.2

175
alertas JPA

55
alertas Mongo

230
alertas Both (raw)

224
identidades únicas Both

Composición exacta

224 esperadas, 224 obtenidas, 0 faltantes y 0 extras al comparar Both con la unión JPA/Mongo.

Compatibilidad QL

48/48 consultas compilaron; Both incluye 24 impactos Call Graph y 7 rutas globales CAST.

Estos recuentos validan los fixtures internos. No equivalen a métricas de precisión/recall sobre proyectos externos ni sustituyen un ground truth.

8 Rendimiento y regresión

Actividad	JPA	Mongo	Both
Consultas analizadas	16	16	16
Database analyze total	218,45 s	218,64 s	217,59 s
Evaluación individual más lenta	6,0 s	2,7 s	2,4 s

Coste acotado

Call Graph se limita a dos saltos y el flujo global parte de un único campo CAST. La evaluación más lenta quedó en 6,0 s en el fixture.

Medición honesta

CodeQL eligió `-Xmx3434M` y 3660 MB off-heap, pero los logs no aportaron RSS máximo real. No se declara una cifra de memoria pico.

Criterios de aceptación de rendimiento

- ✓ Separar compilación y evaluación para no atribuir el coste al predicado equivocado.
- ✓ Ejecutar los perfiles sobre la misma base y hardware.
- ✓ Comparar alertas junto con tiempo: una query rápida que pierde positivos no es válida.
- ✓ Rechazar degradaciones extremas o impredecibles.
- ✓ Preservar regex estáticas y limitar fuentes/sinks antes de añadir propagación.

Los tiempos son observaciones de una ejecución local sobre fixtures internos, no garantías para aplicaciones de mayor tamaño.

9 Extensiones V3.2 validadas

Las dos extensiones se incorporan automáticamente por SCO, sin introducir flags públicos nuevos.

CALL GRAPH · VALIDADO

Delete Relationship / Feature / Move

- Dependencia directa y wrapper que consume el resultado.
- Modificación indirecta de colección.
- Uno y dos niveles, sin cierre transitivo general.
- Overloads, herencia, despacho virtual y helpers privados.
- Negativo: llamada cuyo resultado se descarta y no tiene efecto relevante.
- Evidencia en mensaje: origen, llamador y profundidad.

GLOBAL DATAFLOW · VALIDADO

Solo CAST Attribute / Reference

- Fuente exacta: campo/getter afectado.
- Argumentos, retornos, variables y llamadas encadenadas.
- Sink ligado al tipo anterior, no cualquier uso.
- Varios parámetros del mismo tipo y referencias similares.
- Negativo: consumidor compatible con el tipo nuevo.
- Regresión: ninguna SCO no-CAST activa flujo global.

24	7	0	0
impactos Call Graph	rutas globales CAST	globales fuera de CAST	extras/faltantes Both

Comparación experimental

Conjunto	Uso	Estado
Fixtures internos	Desarrollo, positivos/negativos y coste incremental.	EN USO
PetClinic / StudAdvice	Conjuntos de desarrollo externos.	NO DISPONIBLES
OpenMRS / DZHW	Evaluación independiente contra ground truth.	RESERVADOS

Métricas externas: `compare-sarif.ps1` calcula TP, FP, FN, precision, recall y F1 por regla y global. Solo deben publicarse cuando existan SARIF y ground truth reales; nunca se estiman.

10 Checklist de cierre

Para cada cambio

- ✓ Test unitario de la plantilla.
- ✓ Fixture positivo y negativo.
- ✓ Generación en jpa/mongo/both.
- ✓ Compilación con CodeQL 2.27.2.
- ✓ Ejecución y comprobación SARIF.
- ✓ Medición separada de coste.

Para una release

- ✓ Default idéntico a JPA.
- ✓ Both igual a unión exacta.
- ✓ EvolutionContext encadenado.
- ✓ Sin duplicados ni cruces de framework.
- ✓ Ground truth intacto.
- ✓ Documentación y limitaciones actualizadas.

Artefactos de evidencia

- `src/test/java/**` : unitarios e integración.
- `src/test/resources/fixtures/**` : oráculo Java positivo/negativo.
- `scripts/verify-v2.ps1` : patrón automatizado de validación integral.
- `scripts/verify-v3_2-results.ps1` : positivos, negativos y unión SARIF V3.2.
- `scripts/compare-sarif.ps1` : métricas contra ground truth.
- `docs/V3_2_TEST_RESULTS.md` y `V3_2_PERFORMANCE.md` : resultados medidos.

Conclusión: la coherencia se garantiza combinando pruebas rápidas de generación con compilación y ejecución reales de CodeQL, un oráculo SARIF sensible a positivos y negativos, y controles de composición y rendimiento. Ninguna capa sustituye a las demás.

Documento generado desde el estado local del repositorio. No se realizó push ni se sustituyeron JAR desplegados.